Quiz5

1.KD-Tree

1.1 Definition

A **KD-tree** (short for **k-dimensional tree**) is a binary tree designed for storing points in a (k)-dimensional space.

It is the multidimensional analogue of a binary search tree:

- In a BST, each node partitions values by a single scalar key.
- In a KD-tree, each node partitions space by one coordinate dimension.

For example, in 2D:

- the root may split by the (x)-coordinate,
- the next level splits by the (y)-coordinate,
- then the next level returns to (x),
- and so on.

1.2 Space partitioning properties

The key idea of a KD-tree is **space partitioning**.

Each node:

- stores one point,
- chooses one dimension as the splitting dimension,
- divides the current region into two subregions.

In 2D:

- splitting on (x = a) creates a **vertical line**,
- splitting on (y = b) creates a **horizontal line**.

This gives the following properties:

Property 1: Recursive partitioning

Each node partitions the current region into two smaller regions, and the same idea is applied recursively.

Property 2: Alternating dimensions

In a standard KD-tree, the splitting dimension alternates by depth:

- depth 0: dimension 0,
- depth 1: dimension 1,
- ...
- depth (d): dimension (d \bmod k).

Property 3: Each subtree corresponds to a spatial region

Every subtree implicitly represents a rectangular region (or, in higher dimensions, a hyper-rectangle).

This is what makes geometric pruning possible.

Property 4: Axis-aligned partitions

KD-trees use axis-aligned splits, which makes them simple and efficient for many geometric queries.

1.3 Building a KD-tree

A common construction strategy is:

1. Choose the current splitting dimension.
2. Sort points by that dimension.
3. Pick the median point.
4. Make that point the root of the current subtree.
5. Recursively build the left and right subtrees.

Using the median tends to keep the tree balanced.

1.4 Range search

A **range search** asks:

> Which points lie inside a given query range?

Examples:

- In 1D: find all points in an interval ([a, b])
- In 2D: find all points inside a rectangle
- In higher dimensions: find all points inside a box / hyper-rectangle

How range search works

At each node:

1. Check whether the node's point lies inside the query range.
2. Determine whether the left subtree's region intersects the query range.
3. Determine whether the right subtree's region intersects the query range.
4. Recurse only into subtrees whose regions could still contain valid points.

Why this is efficient

If a subtree's region does **not** intersect the query range, that entire subtree can be pruned.

That is the core geometric advantage of a KD-tree.

1.5 Nearest neighbor search

A **nearest neighbor search** asks:

> Given a query point (q), which stored point is closest to (q)?

Basic idea

A brute-force solution checks all points and takes (O(n)) time.

A KD-tree speeds this up by pruning subtrees that cannot possibly contain a better answer.

How nearest neighbor search works

At each node:

1. Compare the query point to the node's splitting dimension.
2. First recurse into the subtree that contains the query point.
3. Update the current best point found so far.
4. Decide whether the opposite subtree must also be searched.

The pruning rule

Suppose the current best distance is (d).

If the distance from the query point to the splitting plane (or more generally, to the opposite subtree's region) is at least (d), then the opposite subtree cannot contain a closer point and can be pruned.

1.6 Runtime intuition for KD-tree queries

Exact runtime depends on:

- the dimension (k),
- how balanced the tree is,
- the data distribution,
- the query itself.

Typical intuition:

- balanced tree height: about (O(\log n)),
- range search: often much faster than scanning all points,
- nearest neighbor: often much faster than (O(n)) in low dimensions,
- worst case: still (O(n)).

1.7 Strengths and weaknesses of KD-trees

Strengths

- Good for geometric queries
- Supports range search efficiently
- Supports nearest neighbor queries efficiently in low dimensions
- Clear recursive spatial structure

Weaknesses

- Can degrade in high dimensions
- Worst-case query time can still be linear
- Performance depends on balance and data distribution

2.Hash Table

2.1 What is a hash table?

A **hash table** is a data structure for implementing a dictionary / map ADT.

It stores key-value pairs and aims to support:

- insert
- find
- remove

in approximately constant time on average.

2.2 Hash functions

A **hash function** maps a key from a potentially very large keyspace into a smaller integer range, usually an array index range.

Conceptually, a hash function usually has two stages:

1. **Hashing** the key into an integer-like value
2. **Compressing** that value into a valid array index

For example:

- for integers, one may use modular arithmetic,
- for strings, one often combines character values into a numeric code and then compresses.

2.3 Properties of a good hash function

A good hash function should have the following properties:

1.Fast computation

It should be efficient to evaluate.

2.Deterministic

The same key must always map to the same hash value.

3.Good distribution

It should spread keys as evenly as possible over the table.

This leads directly to the idea of **SUHA**.

2.4 SUHA

SUHA stands for **Simple Uniform Hashing Assumption**.

It is the assumption that:

- each key is equally likely to hash to any table position,
- independently of where other keys hash.

This is an idealized mathematical assumption used for runtime analysis.

In practice, no real hash function is perfect, but a good one should behave *approximately* like SUHA for the expected input distribution.

2.5 Hash table array

A hash table is usually implemented on top of an **array**.

Why an array?

- It provides (O(1)) index access.
- A hash function can map a key directly to an array location.

So a hash table usually consists of:

1. a set of keys,
2. a hash function,
3. an underlying array.

2.6 Load factor

The **load factor** is one of the most important quantities in hash table analysis.

It is usually defined as:

[

\alpha = \frac{n}{N}

]

where:

- (n) = number of stored elements,
- (N) = number of table slots.

Interpretation:

- small (\alpha): table is sparse,
- large (\alpha): table is crowded.

The load factor strongly affects runtime.

2.7 Hash table collisions

A **collision** occurs when two different keys hash to the same array location.

Because the keyspace is usually much larger than the array, collisions are unavoidable in general-purpose hashing.

So a hash table must include a **collision-resolution strategy**.

2.8 Open Hashing vs. Closed Hashing

Open Hashing

Also called **separate chaining**.

- Each array entry stores a secondary structure, usually a linked list.
- All keys hashing to the same slot are stored in that bucket.

Closed Hashing

Also called **open addressing**.

- All elements are stored directly in the array.
- When a collision happens, the algorithm probes for another empty slot.

So the naming is confusing:

- **open hashing = separate chaining**
- **closed hashing = open addressing**

2.9 Separate chaining

In **separate chaining**:

- each table index points to a chain (often a linked list),
- colliding elements are appended to the chain.

Advantages

- Simple conceptually
- Easy deletion
- Load factor can exceed 1

Disadvantages

- Extra pointer overhead
- Poor cache locality compared with array-only methods

Runtime intuition

Under SUHA:

- expected chain length is about (\alpha),
- insert, find, remove are (O(1+\alpha)) on average.

If (\alpha) is kept constant, this is (O(1)) average time.

2.10 Linear probing

Linear probing is a closed-hashing strategy.

If the home slot is occupied, check:

- (h(k))
- (h(k)+1)
- (h(k)+2)
- ...

modulo the table size.

Formula

[

h(k,i) = (h(k) + i) \bmod N

]

Main issue: primary clustering

Once a contiguous block of occupied cells forms, it tends to grow, increasing future collision cost.

2.11 Double hashing

Double hashing is another closed-hashing strategy.

It uses two hash functions:

- (h_1(k)): home position
- (h_2(k)): step size

Formula

[

h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod N

]

This spreads probes more evenly than linear probing and reduces clustering.

Advantages

- Better distribution of probe sequences
- Less clustering than linear probing

Constraint

The step size must work correctly with the table size so that probing can eventually visit all necessary slots.

2.12 Re-hashing

Re-hashing is the process of creating a new, usually larger, table and inserting all existing elements into it again using the hash function.

Why re-hash?

- because the table becomes too full,
- because runtime degrades as the load factor grows.

Typical strategy:

1. monitor the load factor,
2. when it exceeds a threshold, allocate a larger table,
3. reinsert all elements into the new table.

This reduces the load factor and restores expected fast operations.

2.13 Runtime of a hash table, in terms of (n)

Strict worst-case runtime for a hash table can be:

- **find:** (O(n))
- **insert:** (O(n))
- **remove:** (O(n))

because all keys could collide.

However, under SUHA and with a controlled load factor:

- average-case **find** is (O(1)),
- average-case **insert** is (O(1)),
- average-case **remove** is (O(1)).

So the practical promise of hash tables is:

- **average-case constant time**, not worst-case constant time.

2.14 Load factor's impact on running times

Load factor is the main driver of performance.

Separate chaining

As (\alpha) grows:

- chains become longer,
- expected search time increases roughly linearly in (\alpha).

Closed hashing

As (\alpha) grows:

- collisions become more frequent,
- probe sequences become longer,
- performance can degrade sharply, especially when (\alpha) gets close to 1.

So in practice:

- keeping the load factor controlled is essential,
- re-hashing is not optional if one wants consistently good performance.

2.15 Properties of a good hash table

A good hash table should have:

1.A good hash function

Keys should be spread evenly.

2.A suitable collision strategy

The collision handling method should match the workload.

3.A controlled load factor

Performance depends heavily on not overfilling the table.

4.Efficient resizing / re-hashing

The structure should recover when performance starts to degrade.

5.Good average-case runtime

The design goal is (O(1)) average insert/find/remove.

6.Practical implementation quality

A good design should also consider:

- memory overhead,
- cache locality,
- deletion cost,
- simplicity of maintenance.
